University of Windsor

01 Course Overview & Modern Database Architecture Part I

Dr. Kalyani Selvarajah
School of Computer Science
University of Windsor

Advanced Database Topics
COMP 8157 01
Winter 2026

Today's Agenda

Course Outlines

Get to know Course Team

Case Study and Discussion

Modern Database Architecture Part I

Why DBMS course?

Jonathan Abrams

Founder of Friendster

Tom Anderson

Co-founder Myspace

Mark Zuckerberg

Co-founder Facebook

“ Friendster was Facebook before Myspace was Facebook before Facebook was Facebook.”

Why DBMS course?

Databases are still **a hot field**.

Most applications have data, and data is usually best stored in a database.

Different approaches to databases for different applications.

Database expertise is a career asset.

JOB Market and Trend!!

Database administrator (DBA) VERIFIED

Posted on December 23, 2022 by Yamin Data Compliance Inc.

Click Here

Job details

161 Bay Street 27th Floor
Toronto, ON
M5J 2S1

82,000 annually for 37.5 hours per week

Permanent employment
Full time

Starts as soon as possible

Benefits: Health benefits, Long term benefits

1 vacancy

Verified

Job Bank #2286361

Salary rate Annual Month Biweekly Weekly Day Hour

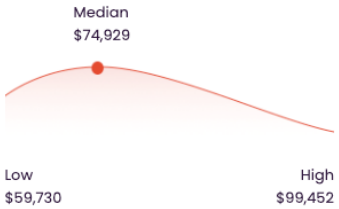
How much does a Database make in Canada?

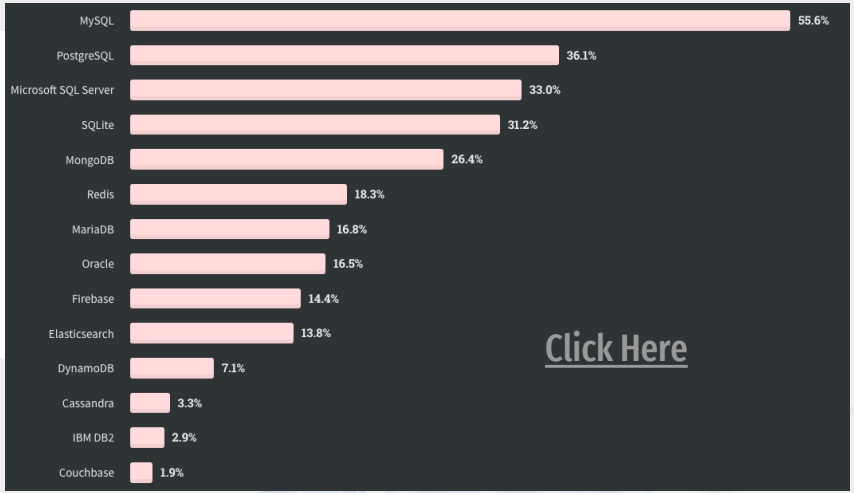
Click Here

$74,929 / Annual

Based on 1344 salaries

The average **database** salary in **Canada** is **$74,929** per year or **$38.43** per hour. Entry-level positions start at **$59,730** per year, while most experienced workers make up to **$99,452** per year.





Click Here

What should you know?

Fundamental DBMS:

3-level architecture,

introduction to file structures: B-trees, B+ Trees and Hashed files,

relational model,

relational algebra and calculus,

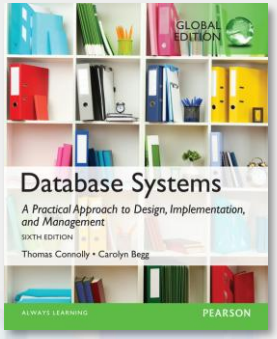
Query languages SQL, PL/SQL, T-SQL

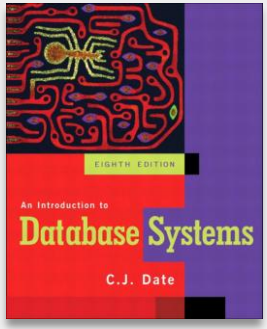
database design with Normalization Theory

Familiar with SQL Server Management Studio (**SSMS**)

Recommended Textbook

Database systems: a practical approach to design, implementation, and management (6th Edition). By T. Connolly and C. Begg.



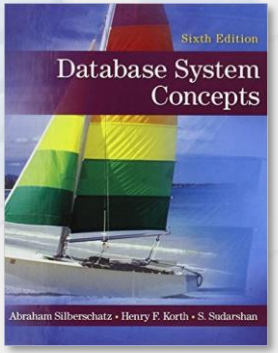


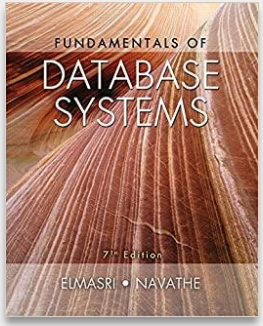
An Introduction to Database Systems (8th Edition).

By C.J. Date

Database System Concepts (6th Edition).

By A. Silberschatz, H. F. Korth, and S. Sudarshan





Fundamentals of Database Systems.

By R. Elmasri & S. B. Navathe

Course Planning

Weeks	Dates	Tentative Topics	Important Dates	
1	05-Jan	Course Overview & Modern Database Architectures Part I	08-Jan	P0: Project group formation **AND** MSSQL & MongoDB environmental setup
2	12-Jan	Modern Database Architectures Part II	15-Jan	Lab 1 AND (12-Jan) P1: Project proposal approval submission *START*
3	19-Jan	Big Data and Modern Data Ecosystems	22-Jan	Lab 2
4	26-Jan	Advanced Data Models & Mining	29-Jan	P1: Project proposal approval submission *END*
5	02-Feb	Distributed Databases: Concepts & Design	05-Feb	Lab 3
6	09-Feb	Distributed DBMS Transparency and Replication	07-Feb 11-Feb	P2: Full Proposal Report Submission AND P3: Project proposal Presentation PPT
7	**16-Feb**	**Reading Week**	**19-Feb**	**No Classes or lab**
8	23-Feb	File organization and indexing	26-Feb	Lab 4
9	02-Mar	Transaction Management	04-Mar 05-Mar	P4: Follow-up milestone Report P4: Follow-up milestone presentation demo
10	09-Mar	Concurrency Control	12-Mar	Lab 5
11	16-Mar	Deadlock and Recovery	21-Mar 22-Mar	P5: Final Report submission P6: Final Project Presentation PPT
12	23-Mar	P6: Final Project Presentation	25-Mar	(Lecture-Tentative Topics) Database Security
13	30-Mar	Courses wrap up	2-Apr	

Note: Students are advised that the schedule and topics described above are tentative and that the material and/or depth and order of presentation are subject to change at the discretion of the instructor and student pace.
This course assumes the student will allocate **a significant amount of independent study and time spent on reading and researching materials as needed**. You are strongly encouraged to ensure sufficient time needed to succeed in this course

Course Policies

For detailed description see course outline

Refer to course outline.

Academic Honesty:

- Refer to UWindsor Policy Page.
- If you're not sure, ask the professor.
- Plagiarism will not be tolerated.

All discussion and announcements will be on Brightspace course page.

Missed Test Make-up:

- The Final Exam for a valid reason, will be allowed to take a makeup exam.

Late Assignment:

- If the assignment is submitted after the due date, you will automatically receive a grade of **ZERO**.

Course Evaluation

Participation in seminars/workshops ……(5%)

Participation in Classroom Activities……(5%)

Labs and Attendance...……..(15%)

~~Assignments ……(15%)~~

Project ……..(45%)

Final Exam……(30%) - TBA

Lab & Attendance

Lab: to understand the topics discussed in the lectures and to learn some advance query environment.

T**ime: 10:00 am to 11:20 am every Thursday**

Lab Exercises (5) have to completed on SQL Server Management Studio (SSMS).

Should complete in the lab, GA will check and Due: at the end of the lab.

Submission: Create a PDF document to provide your answer + SQL file.

Participation in Classroom Activities

Participation in Activities:

Actively participate in the classroom activities

In class Unannounced Quizzes

~~Assignments~~

~~**Assignment 1: MongoDB and Rest API due September 28, 2025, before 11:30 pm.**~~

~~**Assignment 2: Advanced SQL query due November 12, 2025, before 11:30 pm.**~~

~~You should meet with the GA and attend the viva session in order to receive full marks.~~

Project

P0: Forming group		Due Jan 8, 2:30 pm
P1: Project Approval Process	(6%)	Due Jan 31, 11:30 pm
P2: Full Project Proposal Report	(6%)	Due Feb 07, 11:30 pm
P3: Project Proposal Presentation	(7%)	Due Feb 8(ppt submission), Must present in the lab on Feb 12
P4: Follow-up Milestone Report		Due Mar 1, 11:30 pm
P4: Follow-up Milestone presentation demo	(5%)	Due Mar 5, in the lab
P5: Final Project Report	(7%)	Due Mar 21, 11:30 pm
P6: Final Project Presentation ppt		Due Mar 22, 11:30 pm
P6: Project Presentation	(7%)	Due Mar 23, in the class
INDIVIDUAL EVALUATION:		
Individual contribution and Progress	(6%)	From Jan 12 to Mar 23
Q&A individual report (for Project Presentation)	(1%)	Due Mar 23, 1:30 pm

NOTE: The project needs to be developed by a group of **7 students (Mandatory).** A group of 8 members is acceptable in an unexpected scenario. Your project should be stored and maintain in a GitHub Repository. Each member should commit their work

P1: Project Approval Process

- **12th January-** P1: Project proposal approval submission START
- **30th January-** P1: Project proposal approval submission END

Report and Presentation Template

- All presentations should be presented using the following template:

  https://www.overleaf.com/latex/templates/university-of-windsor/kzkzwbsxmjxs

- All Report should be submitted using the following template:

  https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhhncsfqn

Class Support

- All lectures will be held at Windsor Hall,
  - Lectures– Monday 11:30AM - 1:20PM at 167 Ferry (Downtown) 101
  - Lab – Thursday 10:00 AM – 11:20 AM at 167 Ferry (Downtown) 101
- Course Website: https://brightspace.uwindsor.ca/
  - Course Outline and learning outcome
  - Lecture Schedule
  - Slides
  - Assessment Schedule

Course Instructor

Welcome to COMP 8157 Advanced Database Topics Course!

I am Kalyani Selvarajah, former PhD student at University of Windsor and Currently working as Assistant Professor.

I did my undergrad and Masters studies from University of Peradeniya, Sri Lanka.

My research interests are Social Network Analysis and Knowledge graphs using AI and Deep learning techniques.

kalyanis@uwindsor.ca

Office hours: Every Monday from 1:30 pm to 2:00 pm at 300, Ouellette and Tuesday 1:00 pm to 3:00pm on teams

Other times can be arranged by appointment (meaning if you are unable to make my office hours to discuss a concept in a given week, you are encouraged to send me an e-mail asking for a different time. Please include a list of proposed times since I do not have access to your schedule.)

Graduate Assistant

Jack Li

M.Sc. Computer Science student at the University of Windsor with an interest in Cybersecurity and AI.

Office Hours: Thursday 12-2PM

Email: li1la@uwindsor.ca

Courses Taught

At University of Windsor,

- Bigdata analytics (Winter 2024)
- Applied topics in AI (From Fall 2022)
- Advanced Database Topics (From Summer 2020)
- Artificial intelligence Concepts (From Fall 2019)
- Artificial Intelligence for Games (Winter 2020)
- Data Structures and Algorithms (Summer 2020)
- Introduction to Software Engineering (Fall 2020)
- Internship/ Project I (From Summer 2021)
- Internship/ Project II (From Fall 2020)

At Sri Lanka Institute of Information Technology (From 2011 to 2016)

- Introduction to Programming (C++)
- Data Structures and Algorithms
- Database Management System
- Web Applications
- Discrete Mathematics

Routines and Rules in the Classroom

Routines :

Mondays: Lectures on a specific Topics

Thursdays: Lab

Rules:

1. Don’t be shy
2. Respect everyone around you
3. If someone else is talking, let’s listen

Student Responsibilities

- ✓ Be **polite and respectful** to the professor, GAs, business representatives, and fellow students.
- ✓ Attend all classes and meetings **on time**, prepared to participate actively.
- ✓ Take **responsibility** for catching up on any missed announcements or instruction.
- ✓ **Avoid plagiarism** in all assignments.
- ✓ Submit all tasks by their **deadlines**.
- ✓ Keep **mobile phones off** or in airplane mode, and use **laptops** only when permitted by the instructor.

Imagine a Smart City

Case Study: Smart City Traffic Management System

A metropolitan city faces heavy traffic congestion due to rapid urbanization. To improve traffic flow, reduce accidents, and optimize public transportation, the city plans to implement a Smart City Traffic Management System (SCTMS). This system will collect, process, and analyze data from millions of sources—roadside sensors, cameras, GPS in vehicles, and even mobile apps. The system has to handle many different kinds of information: personal details of registered vehicles, real-time traffic flow, accident reports, and maps of the city's road network. It must also process huge volumes of live updates every second to predict congestion and suggest alternate routes. The platform is spread across multiple city zones so that even if one control center goes down, the rest of the system continues working smoothly, and drivers still see a single, consistent view of the city's roads. To make decisions quickly, the system relies on specialized ways of storing and retrieving information—for example, instantly finding the nearest available ambulance during an emergency. At the same time, it must coordinate many overlapping operations, such as closing a road for repairs while rerouting buses and ensuring no conflicts occur. If errors or conflicts do arise, the system detects and resolves them without stopping citywide services. Since it handles sensitive personal and location data, strong protection, access control, and monitoring are essential. Finally, the project team continuously reviews the latest research and innovations to improve the system's scalability, reliability, and ability to adapt to future challenges.

Discussion

1. The system receives information from cameras, sensors, vehicles, and apps. How should the city organize these very different kinds of information so they can work together?
2. Traffic conditions change every second. How should the system keep up so drivers and emergency teams always see the latest updates?
3. If one traffic control center goes down, how can the rest of the system continue to operate without interruption?
4. During an emergency, how can the system quickly identify the nearest available ambulance or hospital?
5. Suppose one team schedules road repairs while another reroutes buses. How can the system prevent these operations from clashing?
6. What could go wrong in such a citywide platform (e.g., network failures, sensor errors)?
7. The platform stores sensitive data like driver identities and live GPS locations. How should the city protect this information from misuse?
8. Technology is always evolving. How can the city keep updating the traffic platform so it remains efficient and reliable in the future?

Case Study: Smart City Traffic Management System

A metropolitan city faces heavy traffic congestion due to rapid [illegible] flow, reduce accidents, and optimize public transportation, the city plans to [illegible] Management System (SCTMS). **This system will collect, process, and analyze data from millions of sources—roadside sensors, cameras, GPS in vehicles, and even mobile apps.** The system [illegible] kinds of information: personal de[illegible] me traffic [illegible]s of the city's road network. It mus[illegible]ve updates every [illegible] to predict congestion and suggest alternate routes. **The platform is spread across multiple city zones so that even if one control center goes down, the rest of the system continues working smoothly,** and d[illegible]sistent view of the city's roads. To make de[illegible] relies on spe[illegible]nd retrieving [illegible] **instantly finding th[illegible]earest available ambulance during [illegible]emergency**. At the same time, it must coordinate **many overlapping operations, such as closing a [illegible]ad for repairs while rerouting buses and ensuring no conflicts occur. If errors or conflicts do arise, the system detects and resolves them without stopping citywide services.** Since it **handles sensitive personal and location data, strong protection, access control, and monitoring are essential**. Finally, the pr[illegible]ct team **continuously reviews the latest research and innovations** to improve the system's scalability, reliab[illegible] and ability to adapt to future challenges.

System Scale and Real-time Updates: big data frameworks, stream processing, indexing, query optimization

Data Collection and Heterogeneous Information : Relational vs document vs graph models; data integrity and quality checks

Distribution Across Multiple Zones: replication, partitioning, distributed transparency, CAP theorem

transaction management, concurrency control, isolation levels

deadlocks, recovery, fault tolerance, rollback, checkpoints

Indexing, spatial queries, query optimization

Security and Privacy

01 Modern Database Architectures

Warm-up & Context

- How many of you have used MySQL or PostgreSQL?
- How many have worked with MongoDB, Neo4j, or Redis?

Evolution of Database

History of Databases - YouTube

1960 1970 1980 1990 2000 2010 2020

IDS

Honeywell - The first network DBMS.

Charles Bachman

1960 1970 1980 1990 2000 2010 2020

IDS

Honeywell - The first network DBMS.

Charles Bachman

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

idms

IDS

Honeywell - The first network DBMS.

Charles Bachman

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

IBM IMS

IDS

Honeywell - The first network DBMS.

Charles Bachman

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

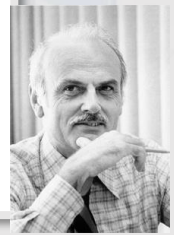
The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. Data Dependencies in Present Systems

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing depend-



Relational Databases

Schema:

Users(user_id, name, email)

Orders(order_id, user_id, product_id, date, amount)

Products(product_id, name, price)

SQL: *Find all orders placed by Alice in the last 30 days?*

```sql
SELECT o.order_id, p.name, p.price, o.date
FROM Orders o
JOIN Users u ON o.user_id = u.user_id
JOIN Products p ON o.product_id = p.product_id
WHERE u.name = 'Alice'
  AND o.date > CURRENT_DATE - INTERVAL '30 days';
```

Relational Databases

ACID:

Atomicity: *When Alice buys a laptop, the system must (1) insert the order and (2) reduce inventory from Products. Both must succeed — not half.*

Consistency: *An order must always reference a valid user_id and product_id. No orphan orders allowed.*

Isolation: *Two users click "Buy" on the last PlayStation at the same time. Only one order should go through.*

Durability: *If the system crashes right after Alice's purchase, the order must still be there when it restarts.*

Which of these guarantees — Atomicity, Consistency, Isolation, or Durability — do you think is the hardest to maintain when millions of users are ordering during Black Friday sales?

Strengths of Traditional RDBMS

1. ACID compliance
2. Structured and Well-Defined Schema
3. Powerful Query Language (SQL)
4. Mature Technology and Ecosystem
5. Security and Access Control
6. Backup, Recovery, and Reliability

That’s why banks, governments, airlines, healthcare, and large enterprises still rely heavily on RDBMS today.

Limitations of Traditional RDBMS

1. Rigid Schema (Low Flexibility)
2. Scalability Challenges
3. Handling Big Data
4. Performance Bottlenecks
5. Limited Support for Unstructured & Semi-Structured Data
6. Cost of Scaling & Licensing
7. Complexity in Distributed Environments

IDS

Honeywell - The first network DBMS.

Charles Bachman

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

ORACLE

Informix TERADATA

IDS

Honeywell - The first network DBMS.

Charles Bachman

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

IDS

Honeywell - The first network DBMS.

Charles Bachman

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, INGRES, Oracle

IDS

Honeywell - The first network DBMS.

Charles Bachman

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

VERSANT

MarkLogic™

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

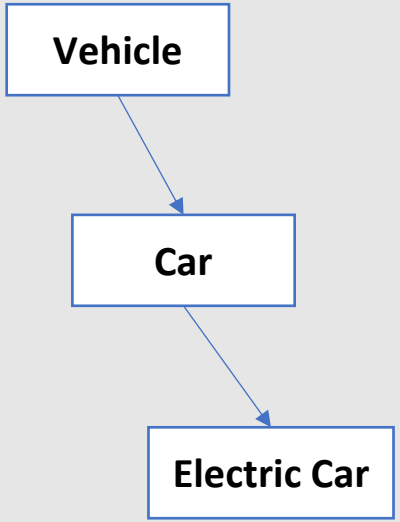
System R, INGRES, Oracle

Why Object-Oriented Databases (OODBMS) are Necessary over RDBMS

1. Solves **object-relational impedance mismatch**.
2. Handles **complex/nested data** naturally.
3. Provides **better integration with OOP languages**.
4. Supports **inheritance, polymorphism**, and object identity.
5. Eliminates overhead of ORMs and complex joins.
6. Suited for domains like **CAD, multimedia, simulations, engineering, scientific databases**.

OODBMS is not a **replacement** for RDBMS — it's **necessary** in contexts where objects and complex data dominate.
Modern practice often uses **hybrid approaches** (e.g., PostgreSQL with JSON support, or Object-Relational Databases).
Industry trend: Instead of separate OODBMS, modern systems integrate object-like storage directly into RDBMS and NoSQL.

Discussion !



This diagram presents a simple

class hierarchy (Vehicle → Car → ElectricCar).

How easy is it to represent this in relational SQL vs. object-oriented DB?

Discussion: In Relational SQL (RDBMS)

Relational databases don't directly support inheritance. To model it, you must choose one of several design strategies:

Single Table Inheritance

One big table with columns for all subclasses.
Unused columns are left NULL.

```sql
CREATE TABLE Vehicles (
    id INT PRIMARY KEY,
    make VARCHAR(50),
    model VARCHAR(50),
    doors INT,
    fuel_type VARCHAR(20),
    battery_capacity INT,
    range_km INT,
    vehicle_type VARCHAR(20)
);
```

Class Table Inheritance

Separate table per class, linked by foreign keys.

```sql
CREATE TABLE Vehicle (
    id INT PRIMARY KEY,
    make VARCHAR(50),
    model VARCHAR(50)
);

CREATE TABLE Car (
    id INT PRIMARY KEY REFERENCES Vehicle(id),
    doors INT,
    fuel_type VARCHAR(20)
);

CREATE TABLE ElectricCar (
    id INT PRIMARY KEY REFERENCES Car(id),
    battery_capacity INT,
    range_km INT
);
```

Concrete Table Inheritance

Only subclasses get their own tables, duplicating parent attributes.

```sql
CREATE TABLE ElectricCar (
    id INT PRIMARY KEY,
    make VARCHAR(50),
    model VARCHAR(50),
    doors INT,
    fuel_type VARCHAR(20),
    battery_capacity INT,
    range_km INT
);
```

Discussion: In Object-Oriented DB (OODBMS)

OODBMS supports classes, inheritance, and objects natively.

You can store objects directly without splitting them into tables.

```
class Vehicle {
   String make;
   String model;
}

class Car extends Vehicle {
   int doors;
   String fuel_type;
}

class ElectricCar extends Car {
   int battery_capacity;
   int range_km;
}
```

```
// Store directly
ElectricCar tesla = new ElectricCar();
tesla.make = "Tesla";
tesla.model = "Model 3";
tesla.doors = 4;
tesla.fuel_type = "Electric";
tesla.battery_capacity = 75;
tesla.range_km = 450;

database.store(tesla);
```

Representing a hierarchy like *Vehicle → Car → ElectricCar* is **cumbersome in SQL (joins, design choices, ORMs)**, but is **natural and straightforward in OODBMS**, since it directly understands inheritance and objects.

IDS

Honeywell - The first network DBMS.

Charles Bachman

OBJECT-ORIENTED DATABASES

In 1983

DB2

IBM– "SEQUEL" becomes the standard (SQL).- **by Donald D. Chamberlin and Raymond F. Boyce**

1960 | 1970 | 1980 | 1990 | 2000 | 2010 | 2020

IDMS

Cullinet – by **Charles Bachman**- primarily a network model (CODASYL) database management system for mainframes.

IMS

IBM - to keep track of the supplies and parts inventory for the Saturn V and Apollo space exploration projects.

Relational Model

IBM– by **Ted Codd-** high level of abstraction of databases to avoid all this sort of maintenance burden on humans.

System R, **INGRES, Oracle**

SQL SERVER

MySQL

PostgreSQL

SQLite

Microsoft SQL Server

MySQL

PostgreSQL

SQLite

Next Week

- (continue) Modern DB architecture
- Into to Bigdata